Horizontal Microprogrammed Control Unit

Component Documentation for the RISCY-STUFF Processor (RISC-201 ISA)

1. INTRODUCTION

The control unit is like the brain of a processor. Every clock cycle, it sends out signals to tell the rest of the processor—the ALU (the math part), the registers, the memory, and the program counter—what to do. In our project, we didn't just hardwire these signals. Instead, we used a "microprogrammed" control unit. This means the patterns of ones and zeros that make up the signals are stored like a mini-program in a small memory block called the control memory. Every time a standard instruction (like an ADD or a LOAD) comes in, the processor runs a tiny sequence of these stored words, called a microroutine, to get the job done.

Specifically, we went with a "horizontal" design. This means every single control signal has its own specific bit in the control word. Because there's no decoding step required between the control memory and the actual hardware parts, a single microinstruction can flip multiple switches on at the exact same time. It makes the control word wider (using more bits), but it’s super fast and flexible. You’ll find all the code for this in `horizontal_control_unit.h` and `horizontal_control_unit.cpp`.

2. HOW THE CONTROL UNIT WORKS

Every instruction goes through a simple three-step process:

- **Fetch and Decode:** First, a built-in two-step sequence runs. It grabs the instruction from memory, moves the program counter forward, figures out what the instruction is, and then points the system to the start of that instruction's specific microroutine.
- **Execute:** The control unit locks in the opcode and sets its own internal pointer (the micro-PC) to the first line of the microroutine. Each line executed tells the micro-PC exactly where to go next, so it just follows the trail.
- **Return to Start:** When a line says the "next" address is the END marker, the routine finishes. The micro-PC resets, ready to fetch the next instruction.

For branching (like "jump if equal"), we have a special test row. If the condition is true, the micro-PC jumps to a "branch_target" address instead of the normal "next" address.

3. CONTROL SIGNALS

Our 42-bit control word is packed with 18 individual control signals and a 5-bit field that tells the ALU what math operation to do. Here is what they all do:

Group	Signal Name	What it does
Front end	**ir_load**	Loads the fetched instruction into the instruction register.
Front end	**instr_decode**	Breaks down the instruction to understand it.
Front end	**opcode_dispatch**	Jumps the micro-PC to the right microroutine for the instruction.
Front end	**operand_read**	Reads the data (registers or immediates) needed for execution.
ALU	**alu_enable**	Turns on the ALU to do some math.
ALU	**alu_src_imm**	Tells the ALU to use a direct number (immediate) instead of a second register.
ALU	**flag_update**	Saves the results of the math (like if it was zero or a negative number) into the FLAGS register.
ALU	**alu_op**	A 5-bit code that tells the ALU exactly what to do (like ADD, SUB, or MUL).
Memory	**mem_read**	Reads data from memory into the processor.

Group	Signal Name	What it does
Memory	**mem_write**	Writes data from a register out to memory.
Memory	**addr_check**	Makes sure the memory address is actually valid before trying to use it. If it's bad, it stops the processor (traps).
Memory	**mem_to_reg**	Tells the processor to save data from memory into a register, rather than data from the ALU.
Register	**reg_write**	Saves the final result into the destination register.
Register	**ra_write**	Saves the return address during a CALL instruction.
Program Counter	**pc_inc**	Moves the program counter forward by one.
Program Counter	**pc_write**	Loads a completely new address into the program counter (used for jumps and branches).
Program Counter	**pc_src_ra**	Tells the program counter to load from the return address register (used for RET).
Control	**halt**	Completely freezes the processor.
Control	**trap**	Signals that an error happened, usually freezing the processor with "halt".

4. MICROINSTRUCTION FORMAT

We packed all of this into a 42-bit word. It's basically split into two jobs: controlling the hardware right now, and deciding where to go next.

Bit Range	Field	Purpose
0 to 7	**Control Low (Bottom)**	Basic datapath controls (alu_enable, mem_read, etc.)
8 to 9	**Control Low (Top)**	Flag updates and halt signals
10 to 14	**alu_op**	Selects what the ALU should do (5 bits)
15 to 17	**cond**	Checks a condition for branching (3 bits)
18 to 25	**next**	The default address for the next step (8 bits)
26 to 33	**branch_target**	Where to jump if the condition is true (8 bits)
34 to 41	**Control High**	Exception handling and front-end fetching (8 bits)

How sequencing works: When a microinstruction finishes, the control unit looks at the "cond" bits. If it's checking for a condition (like "is the number zero?") and that condition is true, the next address is taken from "branch_target". If not, it just uses the normal "next" address. If the next address is 0xFF, it knows the routine is over.

5. CONTROL MEMORY STRUCTURE

The control memory is just a flat array created when the processor starts up. It has room for 255 rows, but we are only using 79 rows right now (which is about 31% full). Every instruction has its own block of rows. A simple lookup table maps standard opcodes to the starting row of their microroutine. If the processor sees an instruction it doesn't recognize, the lookup table automatically points it to row 0, which is an error handler (the ILLEGAL trap).

6. CONTROL MEMORY MAP

Here is exactly where every instruction lives in our control memory:

Row Addresses	Routines	What happens in these rows
0	ILLEGAL (trap)	Unknown instructions land here and freeze the system.
1 to 21	ADD, SUB, AND, OR, XOR, SHL, SHR	Standard math. 3 rows each: grab data, do math, write it back.
22 to 28	LOAD, STORE	Calculates memory addresses and safely reads/writes.
29 to 31	JMP, HALT, NOP	Single rows for jumping, stopping, or doing nothing.
32 to 34	BEQ	Checks if numbers were equal to branch.
35 to 41	MOV, CALL, RET	Moves data, saves return addresses, and jumps back.
42 to 52	NOT, SAR, MUL, CMP	More math. CMP only takes 2 rows because it doesn't write back a result.
53	TRAP	If math fails (like dividing by zero), the system jumps here.
54 to 59	DIV, REM	Division math. Takes 3 rows each.
60 to 62	BGT	Checks if a number was greater to branch.
63 to 76	FADD, FSUB, FMUL, FDIV, FCMP	Floating-point math. These don't update standard integer flags.
77 to 78	FETCH, DECODE	The shared starting point for every single instruction.
79 to 254	Free Space	Empty rows for future features.
255 (0xFF)	END Marker	Tells the system the routine is done.

7. MICROINSTRUCTION TYPES

To save space and keep the code clean, we don't write custom rows for every single instruction. Instead, we use reusable blocks. For example, almost every math instruction uses the same "OPERAND" row to grab its data, and the same "WRITE-BACK" row to save it.

8. INSTRUCTION FUNCTIONS (MICROROUTINES)

Here is a detailed breakdown of all the instructions implemented in the processor and the exact steps (micro-operations) they take to run. Note that every single instruction also takes 2 initial front-end cycles (FETCH and DECODE) before running these steps.

Instruction(s)	Execution Steps (Micro-rows)	Function Description
ADD, SUB	OPERAND → ALU → WRITE-BACK	Standard integer addition and subtraction. Updates hardware flags.
AND, OR, XOR	OPERAND → ALU → WRITE-BACK	Bitwise logic operations. Updates flags.
SHL, SHR, SAR	OPERAND → ALU → WRITE-BACK	Shift Left, Logical Shift Right, and Arithmetic Shift Right.
NOT	OPERAND → ALU → WRITE-BACK	Bitwise complement / inversion.
MUL	OPERAND → ALU → WRITE-BACK	Integer multiplication.
DIV, REM	OPERAND → ALU → WRITE-BACK	Integer division and remainder. Traps if trying to divide by zero.
MOV	OPERAND → MOV → WRITE-BACK	Simply copies a register or immediate number into the destination register.
CMP	OPERAND → ALU (CMP)	Compares two operands and updates flags, but throws away the result (no write-back row).
LOAD	OPERAND → ALU Immediate → MEM READ → WRITE-BACK (memory)	Calculates the base + offset address using the ALU, safely reads from memory, and saves the data to a register.
STORE	OPERAND → ALU Immediate → MEM WRITE	Calculates the base + offset address and writes a register's data out to memory.

Instruction(s)	Execution Steps (Micro-rows)	Function Description
JMP	JUMP	Unconditionally loads a new address into the program counter.
BEQ	TEST (Zero Flag) → NOP or JUMP	Looks at the Zero flag. If set, jumps to the target address. Otherwise, falls through and does nothing.
BGT	TEST (Greater Than Flag) → NOP or JUMP	Looks at the Greater Than flag. If set, jumps to the target address. Otherwise falls through.
CALL	OPERAND → RA WRITE → JUMP	Saves the current program counter into the Return Address (RA) register, then jumps to the subroutine.
RET	PC FROM RA	Restores the program counter from the Return Address (RA) register to return from a subroutine.
FADD, FSUB, FMUL, FDIV	OPERAND → ALU (keep flags) → WRITE-BACK	Floating-point arithmetic. Importantly, this skips updating the integer flags so standard conditions aren't accidentally overwritten.
FCMP	OPERAND → ALU (FCMP)	Floating-point compare. This explicitly sets the integer flags so branches can be used right after.
NOP	idle	Wastes one cycle doing absolutely nothing.
HALT	HALT	Completely stops processor execution.

9. EXCEPTION HANDLING (STAYING SAFE)

We built two safety nets into the control memory to catch errors. First, if a totally invalid instruction is fetched, the system immediately jumps to row 0, flags an "ILLEGAL_INSTRUCTION", and halts the processor.

Second, we handle runtime faults like dividing by zero or trying to access bad memory. The processor only cares about a fault if it happens while that specific hardware is being used (e.g. it only cares about divide-by-zero during an ALU cycle). If a fault happens, the control unit throws away the current signals so it doesn't accidentally corrupt data, logs the error, and forces a jump to row 53 (the TRAP row) to safely freeze everything.

10. SUMMARY

In short, we built a 42-bit horizontal microprogrammed control unit that doesn't need complex decoders. It uses 18 simple control signals and a 5-bit ALU code to run 28 different instructions securely. It fits cleanly into just 79 rows of control memory (out of 255), uses self-contained sequencing to jump around, and has built-in hardware protection to catch bad memory accesses and illegal math before they can crash the system.